Portfolio Milestone Module 4: Results Analysis

Alexander Ricciardi
Colorado State University Global
CSC507: Foundations of Operating Systems
Professor: Dr. Joseph Issa
June 7, 2026

Large text-based data transformations remain common in modern computing because many real systems must read, convert, and rewrite records continuously rather than occasionally. Financial exports, logistics updates, sensor logs, and government transaction files can all grow to millions of rows quickly. Once a workload reaches that scale, performance becomes an operating-systems issue as much as a programming issue. The operating system must coordinate file I/O, buffering, memory use, process behavior, and storage access so that the data can be processed efficiently.

In this project, a one-million-line input file named file1.txt was read and transformed into newfile1.txt, where each output value equals the input value multiplied by two. The Bash baseline processed the file with a line-by-line shell loop, while the Python benchmark compared three required methods: full-file reading, line-by-line streaming, and split-file reading. The final benchmark evidence was captured on Ubuntu 24.04.4 LTS on the `ai01` machine. The system evidence shows an x86_64 platform with 32 logical CPUs, a 13th Gen Intel Core i9-13900KS processor, 125 GiB of RAM, and NVMe solid-state storage. The command-output evidence and benchmark_results.csv show the measured runtimes, the verified output counts, and the sample-value checks.

It is important to note that the same file-transformation task can behave very differently depending on how it is implemented. A shell loop, a full-memory Python pass, a streamed Python loop, and a split-memory Python pass all place different demands on the runtime and the operating system. The benchmark results in this project support that point clearly. In particular, the results show that keeping the work inside one Python process helps substantially, but reading more data into memory is not automatically the fastest design.

TRANSFORMATIONS OF LARGE FILES IN

One common example of a large file transformation is a payroll or billing export. A system may need to read source values, apply a consistent arithmetic rule, and write a new file for downstream processing. Even a simple transformation can become expensive when it is repeated one million

times. Processing speed matters because delays can affect reconciliation, reporting, and scheduled data delivery.

A second example is an industrial or utility data pipeline. Meter readings, sensor measurements, or monitoring counters are often collected in plain-text or CSV-like formats before they are normalized or scaled. In these cases, the arithmetic may be simple, but the repeated file reads and writes still create a meaningful systems workload. Performance matters because slow transformations can delay dashboards, alerts, and operational decisions.

A third example is a public-sector or administrative record system. Large text files may be rewritten to apply format changes, value adjustments, or policy rules before the data is imported into another stage of processing. When the file is large, even a modest per-line cost becomes significant in the total runtime. In other words, large-file transformation performance affects both technical efficiency and practical throughput.

BENCHMARK DESIGN

The Bash baseline was designed to satisfy the assignment model by reading one source line at a time, doubling the value with shell arithmetic, appending the result to newfile1.txt, and reporting elapsed time with the Bash SECONDS variable (GNU Project, n.d.). This method is simple and correct, but it performs the workload through repeated shell-level reads, arithmetic evaluations, and append operations.

The Python benchmark was designed to test multiple implementation strategies on the same one-million-line workload. The program measures elapsed time with time.perf_counter(), verifies the generated output after the timed portion of each run, and records the results in benchmark_results.csv (Python Software Foundation, n.d.). The benchmark compared three Python methods:

- full_read: reads the entire source file into memory, builds the doubled output text, and writes the result.
- line_by_line: reads one source line at a time and writes one doubled output line at a time through Python's buffered file handling.
- split_read: counts the lines, reads the first half into memory, processes it, then repeats the process for the second half.

The common workload contained 1,000,000 input lines, and the output file size was 5,830,325 bytes. That shared workload makes it possible to compare the effect of streaming, full-memory processing, and split-memory processing without changing the size of the task itself.

COMPARATIVE RUNTIME ANALYSIS

Table 1
Measured Runtime Results for the Module 4 Doubling Methods

Method	Output file	Runtime
Bash line-by-line loop	`newfile1.txt`	`8.0000000` seconds
Python full_read	`newfile1.txt`	`0.201569` seconds
Python line_by_line	`newfile1.txt`	`0.195820` seconds
Python split_read	`newfile1.txt`	`0.224031` seconds

Note: The Bash timing and the Python benchmark timings were captured from the Ubuntu target run shown in full_run.png, verification.png, and benchmark_results.csv.

Table 2
Relative Performance Comparisons

Comparison	Result	Interpretation
Python line_by_line vs. Bash	`40.85x faster`	Staying inside one Python process removed much of the shell-loop overhead
Python full_read vs. Bash	`39.69x faster`	Full-memory processing was still far faster than the shell loop
Python split_read vs. Bash	`35.71x faster`	Even the slowest Python method outperformed the Bash baseline decisively
Python line_by_line vs. full_read	`2.85% faster`	Streaming avoided some extra memory-allocation and string-building work
Python split_read vs. line_by_line	`14.41% slower`	The extra counting pass and two-stage processing added overhead

Note: The x multipliers were calculated by dividing the baseline Bash runtime by the corresponding Python runtime. The percentages (%) were calculated relative to the baseline of the targeted method (The 2.85% 'faster' was calculated using. (Full-Read -Line by Line)/Full Read, the 14.41% 'slower' was calculated using (Split Read - Line by Line)/ Line by Line.

The benchmark results show a clear ranking. The Bash baseline was the slowest method at 8.000000 seconds. This result is not surprising because the shell performs repeated loop work and repeated append behavior at a much higher overhead level than Python. The three Python methods completed the same one-million-line transformation in well under one second, which shows that implementation choice matters even when the arithmetic itself is trivial.

The fastest method was Python line_by_line, which completed the workload in 0.195820 seconds. This is an important result because many students would reasonably expect a full-memory strategy to win automatically. In this benchmark, however, the streamed Python loop gave the best overall result. The method kept memory use modest, relied on Python's file buffering, and avoided building one large output string before writing the file.

The full_read method was a close second at 0.201569 seconds. That is still a strong result, and it confirms that the input file was small enough to fit into memory easily on the test system. Even so, the method still had to allocate the entire input text, construct the full transformed output, and then write that large string back to storage. Those extra memory and string-construction steps appear to explain why it finished slightly behind the streamed method.

The split_read method was the slowest Python strategy at 0.224031 seconds. This result also makes sense. Splitting the file into two halves did not create parallel execution; it only changed the order in which memory was used. The method first counted the lines so that it could determine the midpoint, then performed two in-memory processing phases. That additional pass over the file, combined with the repeated list and string handling, made it slower than both other Python methods.

The final command-output evidence also confirmed correctness. The benchmark reported 1,000,000 verified output lines, and the sample values matched the expected doubling rule:

file1.txt: 30106 21507 29865 7056 9623
newfile1.txt: 60212 43014 59730 14112 19246

This verification matters because performance claims are only useful when the generated output is correct.

OPERATING-SYSTEMS INTERPRETATION

The benchmark results in this project are best understood as an operating-systems comparison rather than only a language comparison. Each method produced correct output, but each method used CPU time, memory, buffering, and file output differently. The Bash baseline depended on repeated shell execution behavior. Python full_read traded higher memory use for fewer explicit read operations. Python line_by_line kept the workflow simple and relied on buffered sequential file access. Python split_read changed the memory pattern without introducing true concurrency.

The Bash and Python results are especially useful to compare because they show that runtime overhead can dominate a simple arithmetic workload. Multiplying an integer by two is computationally inexpensive. Most of the measured difference therefore comes from how the runtime reads the file, parses each line, stores intermediate data, and writes the output. In this project, the main gain came from reducing shell-level overhead rather than from adding complexity.

The three Python methods also show that memory strategy and performance are not identical concepts. Reading the entire file into memory can be effective, but it is not always the best result. Splitting the file can sound more structured, but structure alone does not reduce runtime if the work still remains serial. The streamed method became the best match for this workload because it balanced simple control flow, low memory pressure, and efficient buffered I/O.

If this system had more CPU power, the runtimes would probably improve, but the results would not be expected to scale in exact proportion to the added hardware. All four methods in this module remain fundamentally serial transformations, and they still depend on file reading, file writing, and per-line parsing. For that reason, design choice is the more important result in this project than raw CPU count alone. A different implementation strategy would likely matter more than modest hardware growth.

CONCLUSION

The results of this project show that the file-doubling methods behaved differently in ways that operating-systems analysis would predict. The Bash baseline was correct but slowest. All three Python methods were dramatically faster. Python line_by_line produced the best result, which shows that a streamed design can outperform a full-memory design when the workload is simple and the runtime's buffered file handling is already efficient. Python full_read remained competitive, while Python split_read was slower because its extra organizational steps did not provide parallelism or a compensating systems benefit.

REFERENCES

GNU Project. (n.d.). *Bash variables*. In *GNU Bash manual*. GNU Project https://www.gnu.org/software/bash/manual/html_node/Bash-Variables.html

Python Software Foundation. (n.d.). *time - Time access and conversions. In Python 3.12 documentation*. Python Software Foundation https://docs.python.org/3.12/library/concurrent.futures.html